Aperture — Technical Architecture (Part 1)

Version: v1.0

Document Type: Technical Architecture

Related Documents

- Product Requirements Document
- Development Roadmap
- System Architecture
- Low-Level Design

---

1. Purpose

This document explains **why Aperture is built using its chosen technologies and engineering practices**.

The System Architecture document answers:

> **How is the system organized?**

This document answers:

> **Why is it organized this way?**

Every major technical decision should include:

- Problem Statement
- Requirements
- Alternatives Considered
- Decision
- Tradeoffs
- Future Evolution
- Interview Discussion Points

The objective is to ensure that every technology in the project exists because it solves a real problem—not because it is fashionable.

---

2. Engineering Philosophy

Aperture follows five engineering principles.

2.1 Product Before Technology

Technology serves product requirements.

A new technology should only be introduced if it provides measurable value in one or more of the following:

- User experience
- Maintainability
- Scalability
- Reliability
- Developer productivity

2.2 Simplicity Before Scale

The project is intentionally designed to evolve.

Instead of beginning with a highly distributed architecture, Aperture starts with a **modular monolith** and introduces complexity only when justified.

This minimizes:

- operational overhead
- deployment complexity
- debugging difficulty
- cognitive load

while preserving a clear migration path.

2.3 Strong Engineering Foundations

Production quality begins long before production traffic.

Engineering practices—including CI/CD, testing, documentation, observability, and security—are treated as core features rather than optional improvements.

2.4 Interview-Oriented Design

Many architectural decisions are selected because they demonstrate sound engineering judgment.

Examples include:

- choosing a modular monolith over premature microservices

- selecting PostgreSQL for relational integrity
- introducing Redis only after identifying caching needs
- evolving search infrastructure in phases

The goal is to showcase thoughtful tradeoffs rather than an excessive technology stack.

2.5 Evolution Rather Than Replacement

The architecture should support incremental evolution.

New capabilities should extend existing systems rather than replacing them whenever possible.

---

3. Technology Selection Philosophy

Technology choices are evaluated using the following criteria:

1. Fitness for the problem
2. Long-term maintainability
3. Community maturity
4. Learning value
5. Production adoption
6. Integration with the rest of the stack
7. Migration flexibility

No technology is chosen solely because it is popular.

---

4. Frontend Technology

Decision

- React
- Next.js
- TypeScript
- Tailwind CSS
- Framer Motion
- TanStack Query
- Zustand

Why React?

Problem

The frontend requires:

- reusable UI
- component composition
- large application scalability
- strong ecosystem support

Alternatives Considered

- Vue
- Angular
- Svelte

Decision

React offers the strongest ecosystem, extensive community support, and excellent compatibility with Next.js.

Tradeoffs

Pros

- Mature ecosystem
- Massive community
- Excellent hiring relevance
- Rich tooling

Cons

- Larger ecosystem to navigate
- Frequent architectural choices

Why Next.js?

Problem

Aperture is content-heavy.

Movie pages should be:

- indexable
- fast

- shareable
- SEO-friendly

Alternatives

- Vite SPA
- Remix
- Astro

Decision

Next.js provides:

- Server-side rendering
- Static generation
- Incremental regeneration
- Image optimization
- Excellent routing

These capabilities align directly with a metadata-driven platform.

Tradeoffs

Advantages

- Excellent SEO
- Faster initial page loads
- Built-in optimizations
- Production-ready ecosystem

Disadvantages

- Slightly steeper learning curve
- More conventions than a pure React application

Interview Discussion

Why not Vite?

Aperture benefits from server rendering and metadata indexing. Those are significantly easier to implement with Next.js than a client-rendered SPA.

Why TypeScript?

Problem

The project will eventually contain:

- hundreds of API contracts
- dozens of reusable components
- complex data models

Runtime-only validation is insufficient.

Decision

TypeScript provides compile-time guarantees, safer refactoring, and improved maintainability.

Tradeoffs

Pros

- Better tooling
- Safer code
- Easier refactoring
- Self-documenting interfaces

Cons

- Additional learning curve
- More upfront typing

Why Tailwind CSS?

Problem

The application requires a highly customized visual identity.

Traditional component libraries would either:

- impose design constraints
- require significant overrides

Alternatives

- Bootstrap
- Material UI
- Chakra UI

Decision

Tailwind enables a custom design system while maintaining consistency through utility classes.

Tradeoffs

Advantages

- Rapid development
- Consistent spacing
- Responsive design
- Design token compatibility

Disadvantages

- HTML can become verbose
- Requires discipline to avoid inconsistent utilities

Why Framer Motion?

Motion is a core part of Aperture's visual identity.

Framer Motion provides:

- declarative animations
- layout transitions
- gesture support
- accessibility considerations

The goal is to create subtle cinematic interactions rather than decorative animation.

Why TanStack Query?

Problem

The frontend consumes server state extensively:

- movie metadata
- reviews
- recommendations
- feeds
- profiles

Managing caching manually would introduce unnecessary complexity.

Decision

TanStack Query provides:

- request caching
- background refetching
- pagination
- optimistic updates
- cache invalidation

This significantly simplifies server-state management.

Why Zustand?

Application state is divided into:

Server State

Managed by TanStack Query.

Local UI State

Managed by Zustand.

Examples:

- theme
- dialogs
- filters
- navigation state

Separating these concerns keeps the architecture simple.

---

5. Backend Technology

Decision

- Python
- FastAPI
- SQLAlchemy
- Pydantic

Why Python?

Problem

Future roadmap includes:

- semantic search
- embeddings
- recommendation systems
- machine learning

Python has the strongest ecosystem in these domains.

Alternatives

- Node.js
- Java
- Go

Decision

Python minimizes friction between backend engineering and AI capabilities.

Tradeoffs

Advantages

- Exceptional AI ecosystem
- Readability
- Large community
- Rich libraries

Disadvantages

- Lower raw throughput than compiled languages

This tradeoff is acceptable because application performance will primarily depend on architecture, caching, asynchronous processing, and database optimization rather than language execution speed.

Why FastAPI?

Problem

The backend requires:

- asynchronous APIs

- automatic validation
- OpenAPI documentation
- modern typing
- high developer productivity

Alternatives

- Django
- Flask
- NestJS
- Express

Decision

FastAPI provides an excellent balance between performance, developer experience, and AI ecosystem compatibility.

Tradeoffs

Advantages

- Async-first
- Automatic OpenAPI generation
- Strong typing
- Excellent performance
- Native compatibility with Python AI libraries

Disadvantages

- Smaller ecosystem than Django
- Fewer built-in batteries

Interview Discussion

Why not Django?

Django provides many built-in capabilities that Aperture does not initially require. FastAPI offers greater flexibility for a service-oriented backend while integrating naturally with asynchronous workloads and future AI pipelines.

---

6. Architectural Pattern

Decision

Modular Monolith

Problem

The project requires:

- rapid iteration
- clean separation
- low operational complexity

Alternatives

- Microservices
- Layered monolith
- Service-oriented architecture

Decision

Adopt a modular monolith with clear domain boundaries.

Why?

It provides:

- simple deployments
- easy debugging
- shared transactions
- lower infrastructure cost

while preserving a migration path to independently deployable services if scale demands it.

Future Evolution

If a domain develops significantly different scaling characteristics—for example, recommendation generation or metadata ingestion—it can be extracted into its own service without requiring a complete architectural rewrite.